TUYỂN TẬP BÀI TẬP LUYỆN THI TIN HỌC

23 bài CSES Problem Set, dịch tiếng Việt

QUY HOẠCH ĐỘNG

1	1	1	1	1	1	1	1
1	2	3	4	5	6	7	8
1	3	6	10	15	21	28	36
1	4	10	20	35	56	84	120
1	5	15	35	70	126	**210**	330
1	6	21	56	126	252	462	792

mỗi ô = ô phía trên + ô bên trái: 84 + 126 = 210

Lê Thành Nam

Đếm số cách | Dãy con và đoạn con | Bài toán cái túi | Quy hoạch động trên lưới

Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu

LÊ THÀNH NAM

Tổng hợp và biên soạn

MỤC LỤC

Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu

Giới thiệu 2

Chương 1. Đếm số cách 3

Bài 1. Tung xúc xắc *(CSES 1633)* 4
Bài 2. Đếm cách trả tiền I *(CSES 1635)* 9
Bài 3. Đếm cách trả tiền II *(CSES 1636)* 13
Bài 4. Mô tả mảng *(CSES 1746)* 17
Bài 5. Đếm tháp *(CSES 2413)* 20
Bài 6. Đếm số *(CSES 2220)* 24

Chương 2. Dãy con và đoạn con 27

Bài 7. Dãy con chung dài nhất *(CSES 3403)* 28
Bài 8. Khoảng cách chỉnh sửa *(CSES 1639)* 32
Bài 9. Dãy con tăng dài nhất *(CSES 1145)* 36
Bài 10. Đếm dãy con tăng *(CSES 1748)* 40
Bài 11. Dãy núi *(CSES 3314)* 44
Bài 12. Trò chơi bốc số *(CSES 1097)* 49

Chương 3. Bài toán cái túi (balo) 53

Bài 13. Ít đồng xu nhất *(CSES 1634)* 54
Bài 14. Xóa chữ số *(CSES 1637)* 58
Bài 15. Cửa hàng sách *(CSES 1158)* 61
Bài 16. Các tổng tiền *(CSES 1745)* 65
Bài 17. Chia hai tập II *(CSES 1093)* 68
Bài 18. Dự án *(CSES 1140)* 71
Bài 19. Thang máy *(CSES 1653)* 75

Chương 4. Quy hoạch động trên lưới 79

Bài 20. Đường đi trên lưới *(CSES 1638)* 80
Bài 21. Đường đi có xâu nhỏ nhất *(CSES 3359)* 83
Bài 22. Cắt hình chữ nhật *(CSES 1744)* 87
Bài 23. Lát gạch *(CSES 2181)* 91

Lê Thành Nam

GIỚI THIỆU

Tài liệu này dành cho ai

Tài liệu gồm 23 bài của mục Dynamic Programming trong CSES Problem Set (cses.fi/problemset), được sắp lại thành 4 chương theo dạng bài: *Đếm số cách, dãy con và đoạn con, bài toán cái túi, và quy hoạch động trên lưới*. Tài liệu phù hợp với học sinh đã biết lập trình cơ bản và đã làm quen với sắp xếp, tìm kiếm nhị phân, đang luyện thi học sinh giỏi cấp tỉnh và các kì thi tương đương.

Mỗi bài gồm hai phần. Đề bài được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc, kèm bảng subtask do tài liệu bổ sung. Hướng dẫn giải đi qua 4 bước tư duy; mỗi bước có code chạy được, có hình minh họa, bài nào cũng kết thúc bằng bảng so sánh các tầng lời giải đo thật trên máy.

4 bước tư duy giải một bài

Bước	Nội dung
Ngây thơ	Ý tưởng tự nhiên nhất, làm đúng như đề mô tả; có code và độ phức tạp.
Bế tắc	Biểu đồ thời gian đo thật, chỉ ra chỗ chậm hoặc chỗ sai, kết thúc bằng một câu hỏi then chốt.
Phá bế tắc	Nhận xét hoặc tính chất giúp vượt qua chỗ tắc, có ví dụ và hình minh họa.
Tối ưu	Code cuối cùng, độ phức tạp và các lưu ý: kiểu dữ liệu, tràn số, đọc ghi nhanh.

Subtask và bảng so sánh các tầng

Đề CSES không chia subtask. Tài liệu bổ sung cho mỗi bài ba mức: Subtask 1 (30%) là ràng buộc nhỏ để cách vét cạn lấy điểm, Subtask 2 (30%) và Subtask 3 (40%) là ràng buộc gốc của CSES hoặc ràng buộc mở rộng, để phân biệt quy hoạch động thông thường với quy hoạch động đã tối ưu về thời gian và bộ nhớ.

Cuối mỗi bài là bảng so sánh: mỗi tầng code chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm. Một tầng chỉ được điểm của subtask nếu cho kết quả đúng trong không quá 1 giây và dùng không quá 512 MB bộ nhớ (ô "lỗi" là chương trình bị dừng vì tràn bộ nhớ hoặc tràn ngăn xếp 8 MB). Thời gian đo bằng g++ 13 (tùy chọn -O2) trên máy chủ Linux, tính cả đọc và ghi dữ liệu; trên máy chấm khác các con số có thể chênh lệch đôi chút.

Sử dụng tài liệu

Với mỗi bài, hãy tự suy nghĩ ít nhất 20–30 phút trước khi đọc hướng dẫn. Khi đọc, dừng lại ở câu hỏi then chốt cuối phần **Bế tắc** và tự trả lời trước khi xem phần **Phá bế tắc**.

Mọi code trong tài liệu đã được biên dịch, chạy đúng ví dụ của đề, và đối chiếu với lời giải tham chiếu trên 150–300 test ngẫu nhiên.

Lê Thành Nam

Chương 1. Đếm số cách

Trọng tâm chương

Quy hoạch động bắt đầu từ một câu hỏi rất đơn giản: bước cuối cùng là gì? Trả lời câu hỏi đó ta được công thức truy hồi; tính các giá trị từ nhỏ đến lớn và ghi lại để không phải tính lặp. Chương này luyện cách chọn trạng thái, thứ tự tính, và cách đếm theo modulo $10^9 + 7$.

Kiến thức cần nhớ

- Ba bước của quy hoạch động. Chọn trạng thái dp[...] mang đủ thông tin; viết công thức từ các trạng thái nhỏ hơn; xác định giá trị ban đầu và thứ tự tính.
- Từ dưới lên hay từ trên xuống. Đệ quy có nhớ dễ viết nhưng có thể tràn ngăn xếp khi sâu tới 10^6 tầng; vòng lặp từ dưới lên thì an toàn.
- Lấy dư khi đếm. Cộng hai số nhỏ hơn MOD thì chỉ cần trừ MOD một lần; nhân thì dùng long long rồi mới lấy dư.
- Thứ tự vòng lặp. Vòng ngoài là tổng thì đếm hoán vị; vòng ngoài là loại phần tử thì đếm tổ hợp.
- Giảm bộ nhớ. Nếu dòng mới chỉ cần dòng ngay trên, chỉ giữ một hoặc hai dòng.

Các bài trong chương

Bài	Tên bài	Mã CSES	Kĩ thuật chính
1	Tung xúc xắc	1633	Truy hồi, lũy thừa ma trận
2	Đếm cách trả tiền I	1635	Đếm có thứ tự
3	Đếm cách trả tiền II	1636	Đếm không thứ tự
4	Mô tả mảng	1746	QHĐ theo giá trị cuối
5	Đếm tháp	2413	Hai trạng thái, tính sẵn
6	Đếm số	2220	QHĐ chữ số

Lê Thành Nam

Bài 1. Tung xúc xắc

CSES 1633 – Dice Combinations Thời gian: 1 giây · Bộ nhớ: 512 MB
Đề gốc: https://cses.fi/problemset/task/1633

Đề bài

Mỗi lần tung xúc xắc được một số từ 1 đến 6. Hãy đếm số cách tung một hoặc nhiều lần sao cho tổng các số tung được bằng n.

Thứ tự các lần tung có ý nghĩa: 1 + 2 và 2 + 1 là hai cách khác nhau. In kết quả theo modulo $10^9 + 7$.

Dữ liệu vào

Một dòng chứa số nguyên n.

Kết quả

Một số nguyên là số cách, lấy dư cho $10^9 + 7$.

Ví dụ

Dữ liệu vào	Kết quả
3	4

Giải thích ví dụ

Bốn cách: 1 + 1 + 1, 1 + 2, 2 + 1 và 3.

Subtask

Subtask	Ràng buộc	Điểm
Subtask 1	$n \le 20$	30
Subtask 2	$n \le 10^6$ (ràng buộc gốc)	30
Subtask 3	$n \le 10^{18}$	40

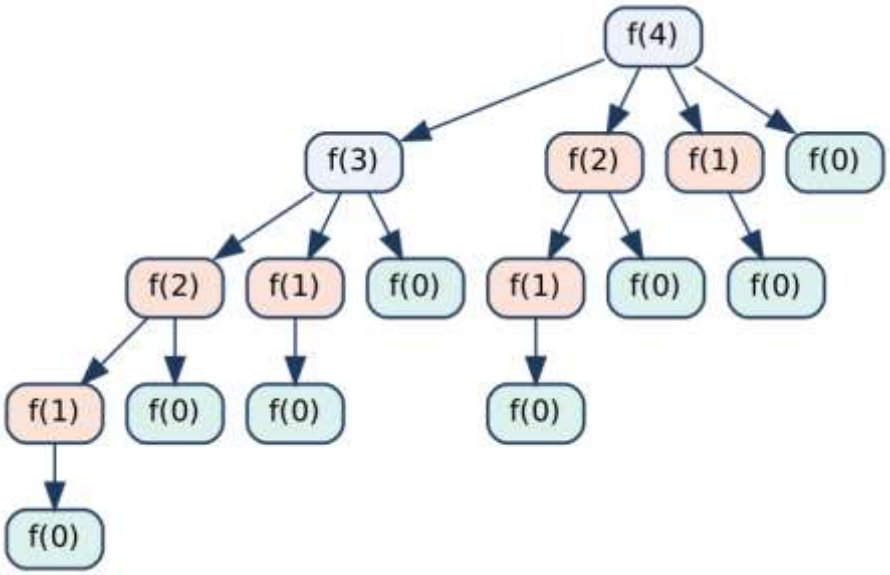
Lê Thành Nam

Hướng dẫn giải

NGÂY THƠ Hỏi lần tung cuối cùng ra mấy

Gọi f(n) là số cách tung để được tổng n. Nhìn vào lần tung cuối cùng: Nó ra một số k từ 1 đến 6, và trước đó các lần tung phải có tổng n - k. Vậy f(n) = f(n - 1) + f(n - 2) + … + f(n - 6), chỉ lấy các số hạng có n - k ≥ 0. Điểm dừng: f(0) = 1, vì tổng 0 có đúng một cách là không tung thêm lần nào.

Kiểm tra với n = 3: f(3) = f(2) + f(1) + f(0) = 2 + 1 + 1 = 4, đúng với ví dụ. Hình 1.1 vẽ các lời gọi khi tính f(4).



Hình 1.1. *Cây lời gọi đệ quy khi tính f(4). Các ô màu cam là những lời gọi bị tính lặp lại.*

Code. Tầng 1. Đệ quy vét cạn

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1e9 + 7;
// f(n) = số cách tung để được tổng n
long long f(long long n) {
    if (n == 0) return 1;                    // đã đủ tổng một cách
    long long ways = 0;
    for (int k = 1; k <= 6; k++)             // lần tung cuối cùng ra k
        if (n - k >= 0) ways = (ways + f(n - k)) % MOD;
    return ways;
}
int main() {
    long long n;
    cin >> n;
    cout << f(n) << '\n';
}
```

Lê Thành Nam

Độ phức tạp: Mỗi lời gọi sinh ra tới 6 lời gọi con, nên số lời gọi tăng theo cấp số nhân, khoảng 2^n. Với n = 20 chỉ vài trăm nghìn lời gọi, chạy rất nhanh.

BẾ TẮC Một giá trị bị tính đi tính lại rất nhiều lần

Hình 1.2 là thời gian đo thật trên test lớn nhất của từng subtask. Với $n = 10^6$ cách đệ quy bị lỗi ngay: Mỗi lời gọi f(n) phải chờ f(n - 1) xong, nên có tới 10^6 lời gọi chồng lên nhau và làm tràn ngăn xếp. Kể cả khi ngăn xếp đủ lớn, số lời gọi khoảng 2^n cũng không bao giờ chạy xong.

Nhìn lại Hình 1.1: Chỉ có 5 giá trị khác nhau f(0), …, f(4), nhưng f(1) bị tính 4 lần, f(0) bị tính tới 8 lần. Câu hỏi then chốt: Có cách nào tính mỗi f(i) đúng một lần không?



Hình 1.2. *Thời gian đo thật của ba tầng lời giải.*

PHÁ BẾ TẮC Phá bế tắc: Tính từ nhỏ đến lớn và ghi lại kết quả

Muốn tính f(i) ta chỉ cần 6 giá trị đứng ngay trước nó. Vậy hãy tính theo thứ tự tăng dần dp[0], dp[1], dp[2], …: Khi tới dp[i], cả 6 giá trị cần dùng đều đã có sẵn (Hình 1.3). Đây chính là quy hoạch động: Mỗi giá trị được tính một lần, các lần sau chỉ việc đọc lại.

Ta còn tiết kiệm được bộ nhớ: dp[i] chỉ dùng 6 giá trị trước, nên chỉ cần giữ 6 số gần nhất trong mảng last, mỗi bước dịch mảng sang trái một ô.



Hình 1.3. *Mỗi ô bằng tổng 6 ô đứng ngay trước nó.*

Code. Tầng 2. QHĐ O(n)

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1e9 + 7;
int main() {
    long long n;
    cin >> n;
    // dp[i] = số cách được tổng i; chỉ cần nhớ 6 giá trị gần nhất
    long long last[6] = {0, 0, 0, 0, 0, 1};    // last[5] = dp[0] = 1
    for (long long i = 1; i <= n; i++) {
        long long cur = 0;
        for (int k = 0; k < 6; k++) cur += last[k];   // dp[i-6] + ... + dp[i-1]
        cur %= MOD;
        for (int k = 0; k < 5; k++) last[k] = last[k + 1];
        last[5] = cur;
    }
    cout << last[5] << '\n';
}
```

Lê Thanh Nam

Độ phức tạp: O(n) thời gian, O(1) bộ nhớ. Với $n = 10^6$ chỉ mất khoảng 0,02 giây, nhưng với $n = 10^{18}$ thì 10^{18} bước là không thể (Hình 1.2).

TỐI ƯU Nhảy nhiều bước một lúc bằng lũy thừa ma trận

Mỗi bước, bộ 6 số (dp[i], dp[i - 1], …, dp[i - 5]) biến thành bộ (dp[i + 1], dp[i], …, dp[i - 4]) theo cùng một quy tắc: Số đầu mới là tổng 6 số cũ, các số còn lại dịch xuống một vị trí. Quy tắc này viết được thành phép nhân với ma trận M kích thước 6 × 6 (Hình 1.4).

Đi n bước nghĩa là nhân với M đúng n lần, tức là nhân với M^n. Lũy thừa nhanh tính M^n chỉ bằng khoảng $2 \log_2 n$ phép nhân ma trận: Bình phương M liên tục và nhân những lũy thừa ứng với các bit 1 của n. Với $n = 10^{18}$ chỉ cần khoảng 120 phép nhân ma trận 6 × 6.

Bộ ban đầu là (dp[0], 0, 0, 0, 0, 0) = (1, 0, 0, 0, 0, 0), nên đáp án dp[n] chính là phần tử ở hàng 0, cột 0 của M^n.

Ma trận chuyển M: (dp[i+1], dp[i], ..., dp[i−4]) = M × (dp[i], dp[i−1], ..., dp[i−5])

1	1	1	1	1	1
1	0	0	0	0	0
0	1	0	0	0	0
0	0	1	0	0	0
0	0	0	1	0	0
0	0	0	0	1	0

Hình 1.4. Ma trận chuyển: hàng đầu toàn số 1 (cộng 6 số), các hàng sau dịch giá trị xuống.

Lê Thành Nam

Code. Tầng 3. Lũy thừa ma trận

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1e9 + 7;
typedef array<array<long long, 6>, 6> Mat;
Mat mul(const Mat &A, const Mat &B) {      // nhân hai ma trận 6 × 6 theo modulo
    Mat C{};
    for (int i = 0; i < 6; i++)
        for (int k = 0; k < 6; k++)
            if (A[i][k])
                for (int j = 0; j < 6; j++) C[i][j] = (C[i][j] + A[i][k] * B[k][j]) %
MOD;
    return C;
}
int main() {
    long long n;
    cin >> n;
    // Trạng thái (dp[i], dp[i-1], ..., dp[i-5]); ma trận M đưa trạng thái i sang i +
1
    Mat M{};
    for (int j = 0; j < 6; j++) M[0][j] = 1;   // dp[i+1] = dp[i] + ... + dp[i-5]
    for (int i = 1; i < 6; i++) M[i][i - 1] = 1;   // dịch các giá trị xuống
    Mat R{};
    for (int i = 0; i < 6; i++) R[i][i] = 1;   // ma trận đơn vị
    for (long long e = n; e > 0; e >>= 1) {     // lũy thừa nhanh: R = M^n
        if (e & 1) R = mul(R, M);
        M = mul(M, M);
    }
```